ONE WITH AI HACKATHON

EvoCharge

Product Design & Team Execution Plan

Nigeria's EV Charging Intelligence Layer — a unified platform for drivers, operators, and policymakers.

EVENT

ONE WITH AI — Arthurite Integrated

PROBLEM STATEMENT

#1 — EV Charging Intelligence Layer

TEAM LEAD

Bashir (Cloud Engineer)

REPOSITORY

GitHub (all collaborators added)

AbdulSamad
Frontend 1 — Driver App

Abdullateef
Frontend 2 — Operator Dashboard

Abdulroheem
Backend Engineer

Bashir
Cloud Engineer & Team Lead

1. What is EvoCharge?

The Problem

Nigeria has no single view of EV charging infrastructure. Drivers, operators, and investors cannot easily answer:

- Where are charging stations?
- Which ones are working?
- What are the wait times?
- Where is demand not being met?

Data sits with individual operators. There is no national intelligence layer.

Our Solution

EvoCharge is a web-based EV charging intelligence platform that:

1. Pulls station data from multiple operators into one place
2. Helps **drivers** find the best charger near them
3. Gives **operators** a dashboard to monitor their network
4. Gives **investors and policymakers** insights into demand and gaps

Who Uses It

User	App	What They Need
EV driver	Driver Web App	Find and choose the right charger
Charging operator	Operator Dashboard	See station status and utilization
Investor / policymaker	Operator Dashboard (Analytics)	See where to invest next

2. MVP Features

Build only what is listed here. Do not add extra scope on hackathon day.

Driver App — AbdulSamad

`apps/driver-web/` `React · TypeScript · Vite · Tailwind CSS`

#	Feature	What It Does
1	Station map	Show chargers on a map, color-coded by status
2	Station list	List view with search and filters
3	Multi-operator view	Stations from 3 operators in one app
4	Filters	Filter by operator, status, connector type
5	Near me	Use device location to find nearby stations
6	Station detail	Show power, wait time, reliability, connectors
7	EvoScore recommend	User enters battery % → show top 3 stations
8	Network Pulse	Map pins update when station status changes
9	Demand heatmap	Toggle overlay showing high-demand areas
10	AI Charge Advisor	Chat box for natural-language charging questions

Operator Dashboard — Abdullateef

`apps/operator-dashboard/` `React · TypeScript · Vite · Tailwind CSS`

#	Feature	What It Does
1	Network overview	KPI cards: total stations, available, utilization, avg wait
2	Status chart	Donut or pie chart of available / busy / offline
3	Operator breakdown	Bar chart showing stations per operator
4	Station table	Table of all stations with live status
5	Demand analytics	Chart of demand by area
6	Peak hours chart	Line chart of busy times
7	Unmet demand panel	Highlight areas that need more chargers
8	Investment summary	Short text block for investors/policymakers

Backend API — Abdulroheem

`backend/api/` `Java 21 · Spring Boot 4.0.6` *(see Section 3)*

#	Feature	What It Does
1	Seed data	25–30 Lagos stations across 3 mock operators
2	Health endpoint	`GET /api/v1/health`
3	Operators API	`GET /api/v1/operators`
4	Stations API	`GET /api/v1/stations` with filters
5	Station detail	`GET /api/v1/stations/{id}`
6	Nearby search	`GET /api/v1/stations/nearby`
7	EvoScore recommend	`POST /api/v1/recommend`
8	Analytics	`GET /api/v1/analytics/summary` and `/demand-by-area`
9	Live events	`GET /api/v1/events/stream` (SSE)
10	AI advisor	`POST /api/v1/advisor`

Cloud / AWS — Bashir

`infra/cdk/` `AWS CDK (Java)`

#	Component	What It Does
1	VPC + security groups	Secure network for all services
2	DynamoDB	Store operators and stations
3	ECS Fargate + ALB	Host the Spring Boot API
4	S3 + CloudFront	Host both web apps
5	EventBridge	Trigger scheduled status updates
6	Amazon Location Service	Map tiles for the driver app
7	Amazon Bedrock	Power the AI Charge Advisor
8	CloudWatch + IAM	Logs, monitoring, permissions
9	Resource tagging	Tag everything with Arthurite APN ID
10	SOW + architecture diagram	Hackathon submission documents

3. Stack Note for Abdulroheem

Action required at kickoff (9 AM): Abdulroheem's listed stack is Python, but the hackathon plan uses **Java 21 + Spring Boot 4.0.6** for the backend.

Option A (recommended): Abdulroheem builds the API in Java/Spring Boot.
Option B: Abdulroheem builds in Python (e.g. FastAPI) — Bashir must adjust ECS/CDK accordingly.

Everyone must agree on one backend language before anyone starts coding.

4. Team Roles and Expectations

Bashir — Team Lead + Cloud Engineer

Owns: `infra/cdk/` , `docs/` , deployment, GitHub merges, check-in calls

Jobs:

- Create the GitHub repo structure and share clone instructions
- Set up AWS account and CDK project
- Deploy all AWS services and share API/CloudFront URLs with the team
- Write the SOW and architecture diagram
- Lead all 4 check-in calls (12 PM, 3 PM, 6 PM, 9 PM)
- Review and merge all pull requests to `main`
- Coordinate the demo video and final submission

Done when: Both apps live on CloudFront, API on ECS, all AWS resources tagged, submission docs ready.

AbdulSamad — Frontend Engineer 1 (Driver App)

Owns: `apps/driver-web/` only

Jobs: Build driver app features (Section 2), connect via `VITE_API_URL` , EV-themed mobile-friendly UI, Framer Motion transitions.

Done when: Driver app runs locally and on CloudFront, demo-ready without errors.

Do not edit: `apps/operator-dashboard/` , `backend/` , `infra/`

Abdullateef — Frontend Engineer 2 (Operator Dashboard)

Owns: `apps/operator-dashboard/` only

Jobs: Build dashboard features (Section 2), charts for utilization/demand/peak hours, clear operator and investor narrative.

Done when: Dashboard runs locally and on CloudFront, demo-ready.

Do not edit: `apps/driver-web/` , `backend/` , `infra/`

Abdulroheem — Backend Engineer

Owns: `backend/api/` and `data/seed/`

Jobs: Publish API contract in hour 1, seed data, all endpoints, EvoScore + advisor, CORS, Dockerfile for ECS.

Done when: All endpoints work, both frontends connected, API runs in Docker on ECS.

Do not edit: `apps/` , `infra/cdk/`

5. Hackathon Day Schedule

Time	Goal	AbdulSamad	Abdullateef	Abdulroheem	Bashir
9–10 AM	Setup	Clone repo, scaffold driver app	Clone repo, scaffold dashboard	Scaffold API, publish API contract	AWS setup, CDK init
10–12 PM	Phase 1	Map + station list	Dashboard layout + KPIs	Seed data + GET endpoints	Deploy DynamoDB + VPC
12 PM	Check-in 1	Demo map	Demo layout	Demo API	Lead check-in
12–3 PM	Phase 2	Detail + recommend + live map	KPIs + station table	Recommend + SSE endpoints	Deploy API to ECS
3 PM	Check-in 2	Demo recommend	Demo table	Demo live pulse	Lead check-in
3–6 PM	Phase 3	Advisor + heatmap + polish	Analytics charts + polish	Analytics + advisor APIs	CloudFront, SOW, diagram
6 PM	Check-in 3	Full driver demo	Full dashboard demo	API stable	Lead check-in
6–9 PM	Submit	Help record video	Help record video	Support live API	Video, upload, submit
9 PM	Deadline	Submit everything			

6. GitHub Collaboration

Repo Structure

```
EvoCharge/
├── apps/driver-web/ → AbdulSamad
├── apps/operator-dashboard/ → Abdullateef
├── backend/api/ → Abdulroheem
├── data/seed/ → Abdulroheem
├── infra/cdk/ → Bashir
├── docs/ → Bashir (team via PR)
└── README.md → Bashir
```

Golden rule: Only edit your own folder. Ask in team chat before touching someone else's code.

Branching

`main` — always working; only Bashir merges here. Everyone else uses feature branches.

Person	Branch Format	Example
AbdulSamad	`feat/driver-*`	`feat/driver-map`
Abdullateef	`feat/operator-*`	`feat/operator-charts`
Abdulroheem	`feat/api-*`	`feat/api-stations`
Bashir	`infra/*`	`infra/cdk-deploy`

Workflow (Step by Step)

1. Get the latest code

```
git checkout main
git pull origin main
```

2. Create your branch

```
git checkout -b feat/driver-map        # AbdulSamad
git checkout -b feat/operator-charts   # Abdullateef
git checkout -b feat/api-stations      # Abdulroheem
git checkout -b infra/cdk-setup        # Bashir
```

3. Work and commit often

```
git add apps/driver-web/
git commit -m "feat(driver): add station map component"
```

4. Push your branch

```
git push -u origin feat/driver-map
```

5. Open a Pull Request

Base branch: `main`. Add a short description. Tag Bashir as reviewer.

6. Bashir reviews and merges

7. Pull main again before new work

```
git checkout main
git pull origin main
```

Commit Message Format

```
feat(driver): add station map
feat(operator): add demand chart
feat(api): add GET /stations endpoint
infra(cdk): add DynamoDB tables
fix(api): fix CORS for CloudFront
```

Avoiding Conflicts

1. Stay in your folder
2. Abdulroheem publishes the API contract first
3. Frontenders use mock data until API is ready
4. Never commit `.env` files — only `.env.example`
5. Pull `main` every 1–2 hours

Resolving Merge Conflicts

```
git checkout main
git pull origin main
git checkout your-branch
git merge main
# fix conflicts in your files only
git add .
git commit -m "chore: resolve merge conflict"
git push
```

If unsure, message Bashir before forcing anything.

What Not to Commit

- `.env` (secrets and API keys)
- `node_modules/`
- `target/` (Java build output)
- `cdk.out/`
- AWS credentials
